



Comparaison between the performance of distinct models in classification of swimming types using rolling windows and compression methods

Nadine Dirani, Guillaume Pelletier

► To cite this version:

Nadine Dirani, Guillaume Pelletier. Comparaison between the performance of distinct models in classification of swimming types using rolling windows and compression methods. 2023. hal-04368009

HAL Id: hal-04368009

<https://hal.science/hal-04368009v1>

Preprint submitted on 31 Dec 2023

HAL is a multi-disciplinary open access archive for the deposit and dissemination of scientific research documents, whether they are published or not. The documents may come from teaching and research institutions in France or abroad, or from public or private research centers.

L'archive ouverte pluridisciplinaire **HAL**, est destinée au dépôt et à la diffusion de documents scientifiques de niveau recherche, publiés ou non, émanant des établissements d'enseignement et de recherche français ou étrangers, des laboratoires publics ou privés.

Comparaison between the performance of distinct models in classification of swimming types using rolling windows and compression methods

Nadine Dirani¹, Guillaume Pelletier¹

¹ Dotvision, 77550 Moissy-Cramayel, France

email: {ndirani, guillaume.pelletier}@dotvision.com

December 31, 2023

Abstract

This article investigates the synergy between Artificial Intelligence (AI) and swim biomechanics, aiming to improve the classification of a variety of swimming styles. Acknowledging the crucial role of precise stroke classification in coaching and performance evaluation, we harness wearable technology-specifically, acceleration and gyroscope data integrated under athletes' caps. Within this paper, our focus centers on diverse methods and models based on 3D accelerometer and gyroscope data. We propose and assess four distinct versions for multi-class classification, employing various time-sequence and compression techniques. We meticulously outline the advantages and drawbacks of each approach. The outcomes reveal a notable accuracy achieved through straightforward methods that are not only swift in computation but also operate at a remarkably low sampling frequency. These findings underscore the efficacy of our approach and its potential to provide reliable insights into swimming biomechanics, leveraging accessible wearable technology for enhanced stroke classification.

1 Introduction

Swimming, beyond being a popular recreational pursuit, stands as a complex and multifaceted sport requiring meticulous analysis for effective coaching, competitive evaluation, and overall performance assessment. The distinct strokes-freestyle, backstroke, breaststroke, butterfly, and individual medleys-each demand a nuanced understanding due to their unique techniques and intricate body movements. The precise classification of these strokes is imperative for enhancing coaching strategies, conducting competitive analyses, and comprehensively assessing swimmers' overall performance. In recent years, the integration of Artificial Intelligence (AI) into the realm of swimming has catalyzed a paradigm shift in stroke classification and biomechanical analysis. AI technologies, notably computer vision and machine learning, have become indispensable tools for automating the identification of different swimming styles. This intersection of AI and swimming not only streamlines the classification process but also contributes significantly to a deeper understanding of the biomechanics inherent in each stroke. The core strength of AI lies in its capacity to categorize input

data into predefined classes based on specific patterns or features, a concept pervasive across various domains, including computer vision, natural language processing, healthcare, and finance. Within the context of swimming, the application of AI-driven classification techniques has become pivotal, reshaping how swimmers' techniques are evaluated and understood. Researchers grappling with the intricate challenge of swim type classification have turned to sophisticated AI techniques, with a notable emphasis on deep learning methodologies. Recurrent Neural Networks (RNNs), a subset of neural network models characterized by a self-connected hidden layer, have emerged as potent instruments for capturing the temporal dependencies intrinsic to swimming movements. The sequential nature of strokes and the dynamic interplay of body movements make RNNs particularly adept at modeling and classifying diverse swimming styles. In the field of AI-driven swim type classification, our study significantly contributes to the existing body of knowledge by exploring the synergistic integration of three key techniques: rolling windows, distillation, and quantization. The utilization of rolling windows addresses the temporal intricacies of swimming movements, acknowledging the dynamic nature of strokes and the variability in swimmers' movements over time. Distillation, involving the transfer of knowledge from a complex model to a simpler one, facilitates the extraction of essential insights from intricate swimming patterns, contributing to a more streamlined classification process. Quantization, focused on compressing the model's parameters, results in a more memory-efficient framework, enhancing the feasibility of real-world applications. As we delve into the intricacies of our study in the subsequent pages, we aim to provide comprehensive insights into our methodology, experimental setup, and results, shedding light on the efficacy of our approach in the context of swim type classification. The convergence of AI and swimming not only unlocks exciting possibilities for the sports and fitness industry but also underscores the broader potential of AI applications in nuanced and dynamic domains. The synthesis of advanced technologies and the profound understanding of swimming biomechanics present in this research set the stage for a new era in swim type classification and performance assessment.

The intersection of wearable technology and swimming biomechanics has garnered considerable attention in recent literature. Studies by [3] and [10] have explored the integration of accelerometers and gyroscopes to analyze the kinematics of various swimming strokes. These investigations highlight the potential of wearable devices in capturing the intricate details of swimmer movements, facilitating a more profound understanding of stroke mechanics.

Furthermore, the use of multiclass classification techniques in swimming analysis has been a subject of interest. In a study by [12], machine learning algorithms were employed to classify swimming styles based on accelerometer data. The findings demonstrated promising accuracy in distinguishing between different strokes, laying the groundwork for the application of advanced classification methods in swim biomechanics.

Our research incorporates both acceleration and gyroscope data for multiclass classification of swimming types. By leveraging a wearable device placed in the cap of athletes, we aim to enhance the accuracy and comprehensiveness of swimming style classification. The integration of gyroscope data adds an extra layer of granularity,

capturing rotational movements and further refining the classification process. This article delves into the innovative intersection of Artificial Intelligence (AI) and swimming biomechanics, with a focus on enhancing the classification of diverse swimming styles. Recognizing the pivotal importance of accurate stroke classification in both coaching and performance analysis, our approach utilizes state-of-the-art wearable technology. Specifically, we integrate acceleration and gyroscope data collection devices under the athletes’ swim caps. This placement is strategic; not only does it align with the natural movements of swimmers, but it is also uniquely suited for signal transmission. The embedded WiFi and Narrowband IoT 5G connectivity of these devices are optimized in this position, ensuring uninterrupted data transmission even while swimming. This setup provides a comprehensive and real-time analysis of swimmers’ performance, marking a significant advancement in the field of sports technology and analytics.

This paper is organized as follows. Section 2 briefly recall wearable device and collection data. Section 3 presents the classification process. In this section, we present the proposed architecture and different versions used to improve our model. Section 4 addresses the experiments and illustrate the results obtained demonstrating the accuracy, efficiency and robustness of the proposed method and model.

2 Data collection

Inertial Measurement Units (IMUs), holds great promise for swim tracking, particularly given the advancements in this technology. IMUs have evolved to the point where they can be compact, lightweight, waterproof, and fully wearable without hindrance to the athlete. Moreover, enhancements in battery life and memory capacity now enable the continuous recording and storage of data for extended periods, with minimal intervention required from coaches or athletes. The application of these advanced IMUs for swim monitoring allows for more precise, consistent, and comprehensive data collection.

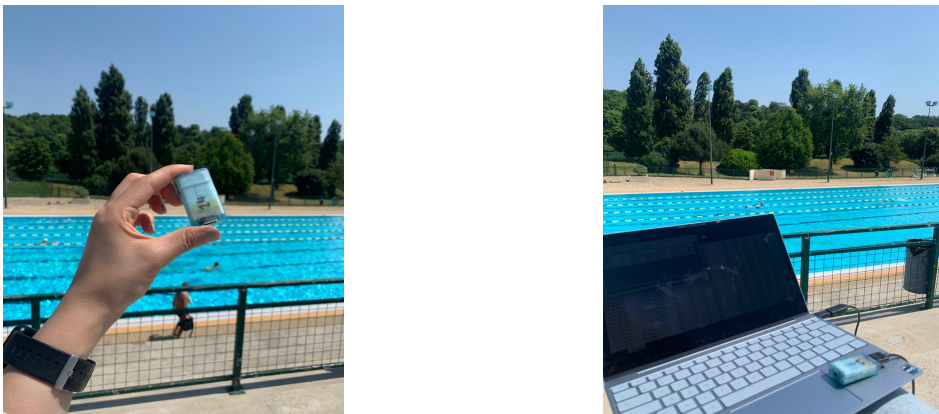


Figure 1: Hardware setup: the wearable device (on the left) and and the transmission of data (on the right).

In this work, we use Lara device (Figure 1), which is a swim activity tracker dedicated to measure swim performance in real time, using IMU, GPS and WIFI/LTE as transmission mode. This repository host all the necessary documents, code and data to explain and build the data flow, from sensor to AI (see Figure 2) .

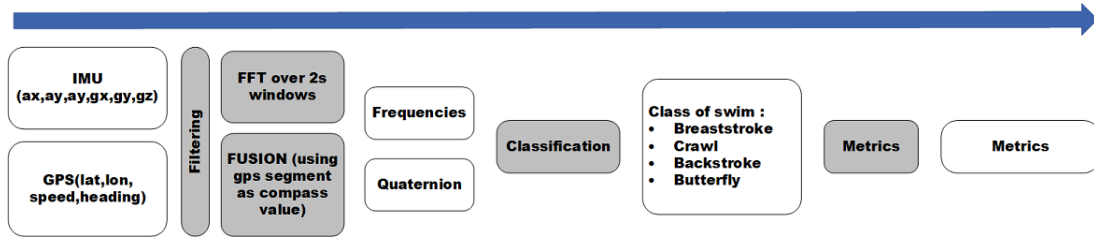


Figure 2: Lara device

Lara device has been designed (ajouter la taille du device avec photo) containing an accelerometer, gyroscope, microcontroller, battery and transceiver. The accelerometer and gyroscope are sampled on three orthogonal axes in order to accurately register the motion of the athlete. The system can therefore register the aforementioned motion data, which is ample for registering data during our preliminary tests. It can be stated that the size of the device does not influence the training process nor the biomechanical environment of the swimmer.

Data from tri-axial accelerometers and gyroscopes was gathered and saved. The test run encompassed a 200m individual medley (IM), as it incorporates all four recognized swimming stroke types, namely butterfly, backstroke, breaststroke, and freestyle.

Lara device is positioned on the back of the head, with the following IMU settings on acceleration and gyroscope as shown in Figures 3 and 4.

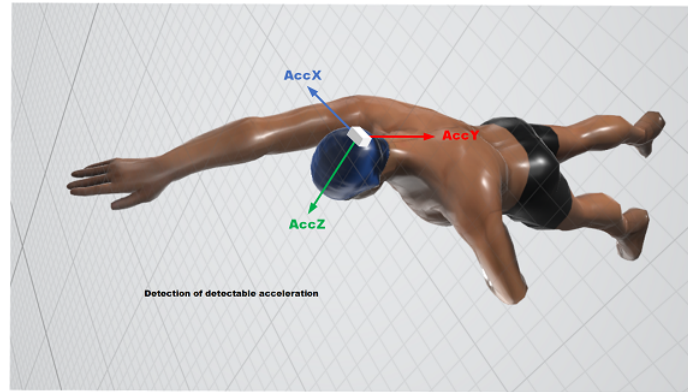


Figure 3: Detectable acceleration

Data was transferred from the Lara device and stored for further analysis as a CSV file under the time series folder with all recorded values, with the following structure as in Figures 5 and 6.

Figures 7 and 8 show this filtered 200m IM data with the module mounted on the head for two different swimmers. From the left to the right, Figures 7 and 8 show first butterfly followed by backstroke then breaststroke and finally freestyle (crawl).

The four separate strokes are clearly distinguishable. Represented by the x-axis is sample number, and on the y-axis is acceleration. The largest spikes in the surge trace (clearly visible) are caused by the athlete pushing off the wall (or a dive start in some cases). Therefore these spikes can be used to identify segment starts and turns, differentiated by the nature of the activity before and after the spike. The four

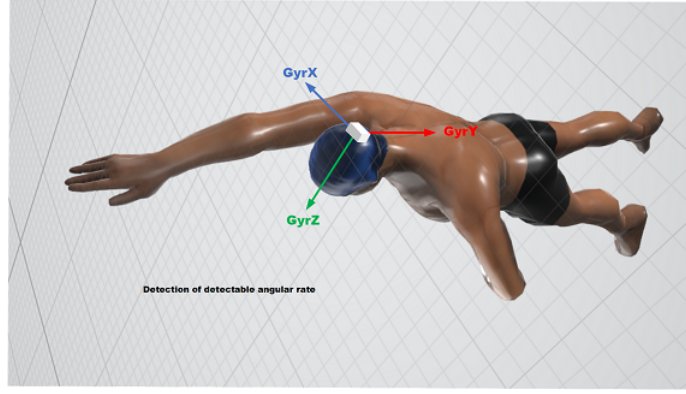


Figure 4: Angular rate

Timestamp	Fix	Lat	Lon	Alt	pAcc	Mag	Speed	sAcc	Heading	Battery	AccX	AccY	AccZ	GyrX	GyrY	GyrZ
-----------	-----	-----	-----	-----	------	-----	-------	------	---------	---------	------	------	------	------	------	------

Figure 5: CSV file

Timestamp	Fix	Lat	Lon	Alt	pAcc	Mag	Speed	sAcc	Heading	Battery	AccX	AccY	AccZ	GyrX	GyrY	GyrZ	Swim_class	
10:24:14.426	False	0	0	0	0	0	0	0	0	0	4,13	837,408	-257,664	-130,296	6370	22470	26950	Out
10:24:14.521	False	0	0	0	0	0	0	0	0	0	4,13	732,488	-327,448	182,024	-23450	-42910	-50050	Out
10:24:14.631	False	0	0	0	0	0	0	0	0	0	4,12625	734,446	-355,996	-218,38	35280	95690	11480	Out
10:24:14.731	False	0	0	0	0	0	0	0	0	0	4,12625	933,056	-423,584	23,912	78330	80010	21350	Out
10:24:14.826	False	0	0	0	0	0	0	0	0	0	4,12875	866,932	-313,296	-340,38	6720	48860	23940	Out
10:24:14.926	False	0	0	0	0	0	0	0	0	0	4,12875	538,508	-333,548	-16,104	11410	43750	138180	Out
10:24:15.026	False	0	0	0	0	0	0	0	0	0	4,12875	564,616	104,676	-530,944	-19040	71400	119840	Out
10:24:15.131	False	0	0	0	0	0	0	0	0	0	4,12625	493,124	98,82	-661,972	-5250	24010	-137760	Out
10:24:15.231	False	0	0	0	0	0	0	0	0	0	4,12625	799,344	-352,58	-890,6	-117810	-43890	-136430	Out
10:24:15.326	False	0	0	0	0	0	0	0	0	0	4,12875	780,8	-173,728	-774,7	27370	22470	30380	Out
10:24:15.426	False	0	0	0	0	0	0	0	0	0	4,12875	665,876	-147,132	-961,848	77420	4060	30660	Out
10:24:15.526	False	0	0	0	0	0	0	0	0	0	4,12875	560,224	-212,28	-1021,384	43050	-17290	39200	Out
10:24:15.631	False	0	0	0	0	0	0	0	0	0	4,1275	325,008	-56,12	-1065,792	17850	-14840	19320	Out
10:24:15.731	False	0	0	0	0	0	0	0	0	0	4,1275	321,592	10,004	-1062,62	-9660	-8750	-13510	Out
10:24:15.826	False	0	0	0	0	0	0	0	0	0	4,1275	381,86	43,676	-1099,708	-21630	-5530	-20930	Out
10:24:15.926	False	0	0	0	0	0	0	0	0	0	4,1275	480,192	-43,92	-1087,264	-10360	-56420	-24640	Out
10:24:16.026	False	0	0	0	0	0	0	0	0	0	4,1275	676,124	-101,504	-753,716	-17640	-85050	-40320	Out
10:24:16.131	False	0	0	0	0	0	0	0	0	0	4,125	596,58	-31,964	-736,392	-9030	-39340	-20860	Out
10:24:16.231	False	0	0	0	0	0	0	0	0	0	4,125	657,824	-65,88	-338,428	20440	50750	11830	Out
10:24:16.326	False	0	0	0	0	0	0	0	0	0	4,125	734,928	-185,684	-409,92	25900	93240	9100	Out
10:24:16.426	False	0	0	0	0	0	0	0	0	0	4,125	749,08	-97,112	-503,616	35630	23470	-1330	Out
10:24:16.526	False	0	0	0	0	0	0	0	0	0	4,125	789,828	-80,276	-511,912	29960	-14770	19670	Out
10:24:16.631	False	0	0	0	0	0	0	0	0	0	4,125	869,372	-33,428	-616,1	-4760	-2590	36330	Out
10:24:16.731	False	0	0	0	0	0	0	0	0	0	4,125	769,088	195,932	-264,252	-17710	-39620	44030	Out

Figure 6: Example of the data acquired by the wearable sensor.

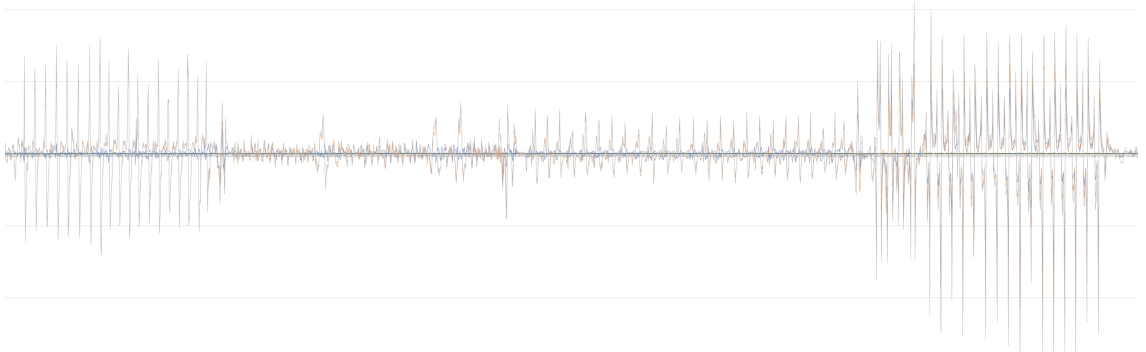


Figure 7: Measured data after filtering of a 200m individual medley in a 50m pool (swimmer 1)

swimming strokes are separately illustrated in Figure 9.

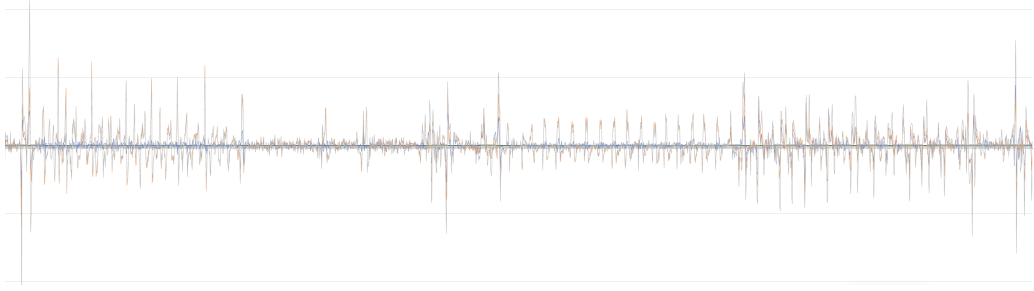


Figure 8: Measured data after filtering of a 200m individual medley in a 50m pool (swimmer 2)

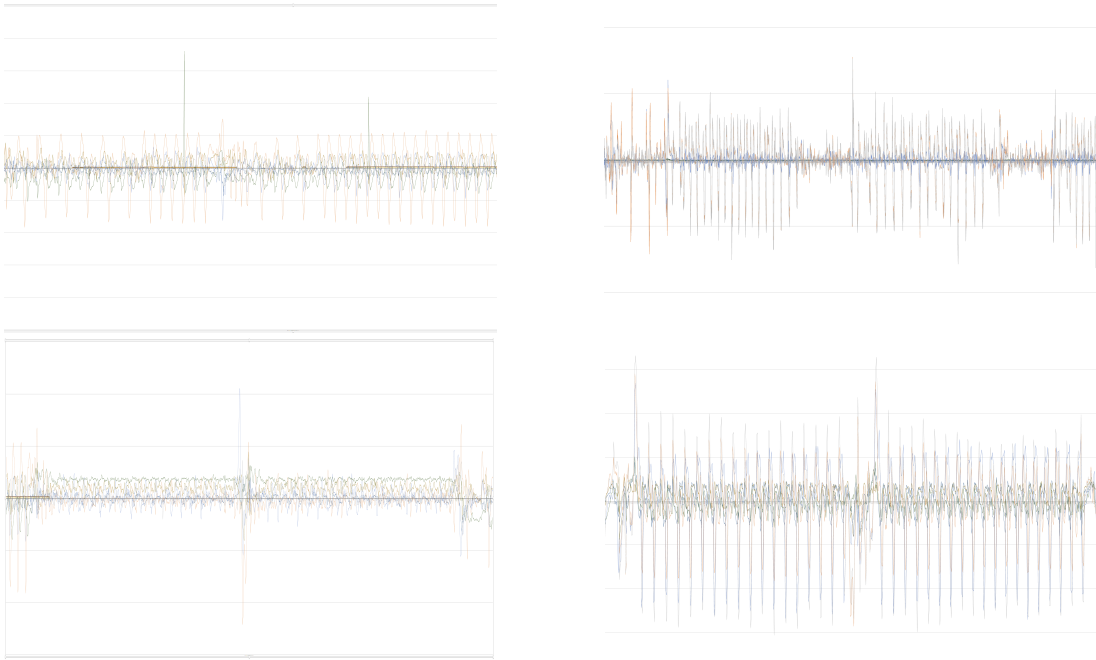


Figure 9: The four swimming strokes starting with butterfly, backstroke, breaststroke and finally crawl.

3 Classification using Long Short-Term Memory Networks

3.1 Recurrent Neural Networks

Recurrent Neural Networks (RNNs) belong to a category of neural network models featuring a self-connected hidden layer. This unique recurrent connection allows the network to maintain an internal state, enabling it to retain past context information. As one might expect, the ability to understand context is absolutely vital for tasks such as sequence recognition and classification.

RNNs can serve different purposes, including sequence classification, sequence generation and sequence creation. Sequence classification can be employed to classify an entire sequence into a single category, making them suitable for tasks like sentiment analysis or speech recognition. RNNs are also capable of generating entirely new sequences as output. For instance, they can be used for translating text from one

language to another. In addition, RNNs are proficient at generating a sequence based on a single input. This is particularly useful, for instance, in the automatic captioning of images.

In summary, RNNs are a versatile class of neural networks equipped with the ability to process sequential data, making them indispensable for various applications where an understanding of context and temporal dependencies is required. The inherent limitation of standard RNNs lies in their restricted capacity to capture extended contextual information, making it challenging to learn long-term dependencies. This constraint arises from the extent to which a given input can impact the hidden layer. The recurrent connection within RNNs results in a scenario where the input’s influence either diminishes or grows exponentially, a phenomenon often referred to as the “ vanishing gradient problem ” as discussed in [8, 13]. This issue holds particular significance for our specific application because it is impossible to determine in advance the length of our sequences. In other words, the duration of specific actions is not constrained or predefined.

The RNN architecture introduced by [7] is known as Long Short-Term Memory (LSTM) and offers a solution to the vanishing gradient problem. Within the LSTM’s hidden layer, there are specialized memory blocks, which can be thought of as self-connected subnetworks, each containing multiple internal cells. What sets LSTM apart is its ability to store and access information over extended sequences, as outlined by [13]. In essence, the LSTM carries data from one time step to another, with each cell at a given time step being capable of both retaining and modifying information as it processes sequential input.

A distinguishing feature of LSTM cells is the inclusion of gates [7]. These gates enable the cells to make decisions regarding which information should be retained and which should be discarded as the sequential processing unfolds. This selective retention or discarding of information is what allows LSTMs to effectively capture and propagate long-term dependencies in data sequences.

3.2 Classification

The swimmer stroke classification problem can be described as a multi-class problem that can be cast as a time-series of swimmer’s positions. The recognition of swimming style are recognized using simple classification methods. The classification was done using the sliding window technique and the appropriate window size was decided based on various tests. Windowing is the predominant and straightforward method for feature representation [4, 5]. This approach utilizes a sliding window that continually incorporates the latest and most current data, disregarding outdated data points. Essentially, this mechanism retains only the most recent data for a specific duration before discarding it. The selection of the optimal window size is a vital consideration. The goal is to retain pertinent past positions for detection while excluding older positions that hold no significance for real-time stroke detection. It was noted that the best classification accuracies for the data were obtained using about 20 seconds long windows with a slide of 2 seconds between two sequential windows. From each window, features were extracted and the classification was done based on these features. The methods presented in this study are efficient; therefore the features used in the classifi-

cation process are simple and fast to calculate, such as standard norm and magnitude, fusion of acceleration, gyroscope and orientation data, in addition, normalization of acceleration. The acceleration, gyroscope and orientation are normalized and fused in an attempt to reduce the artefacts when the low acceleration assumption does not hold.

3.3 Proposed Network Architecture

The proposed architecture for swim classification is designed with a detailed framework. This architecture is a sophisticated neural network configuration designed to process sequential data with a comprehensive set of features. It begins with an input layer that receives data in the form of sliding windows, each containing 16 distinct features. These sliding windows enable the model to capture temporal dependencies within the data. To handle the variability in input sequences, sliding windows with a window size of 20 and a stride of 2 are employed, ensuring that the model can effectively process diverse input lengths. The architecture includes two LSTM layers, which are specialized for modeling sequences, and two dense layers that facilitate feature extraction and transformation. The LSTM layers play a pivotal role in capturing long-term dependencies in the data, making it suitable for tasks involving complex temporal patterns. In addition, the sliding windows technique is employed to ensure that the model can process data sequentially, preserving the temporal context. To enhance efficiency, quantization is incorporated to reduce the complexity of the model while preserving essential information.

Compression through quantization involves mapping a large input set to a smaller output set, effectively grouping weights and activations with similar values to reduce the number of independent parameters. Hash-net [2] imposes constraints on weights by hashing them into distinct groups prior to training. Within each group, weights are shared, and only the shared weights and activations and hash indices need to be stored. Another approach by [6] compresses networks using vector quantization techniques, but these quantization methods introduce a time overhead and can impact network performance to some extent.

More recently, [14] introduced the Incremental Network Quantization (INQ) method. This approach divides the weights and activations into two separate parts: one for quantization and the other for retraining to compensate for quantization-induced loss. It enables gradual quantization of network weights, resulting in a quantized model with accuracy that can even surpass that of the original model. Essentially, this method effectively addresses the issue of accuracy loss during network compression.

A distinctive aspect of this architecture lies in its incorporation of distillation. Deep neural networks impose significant resource demands, limiting their suitability for environments with restricted bandwidth and power—particularly on edge devices equipped with modest GPUs and limited battery capacities. In our case, we don’t have access to GPUs or CPUs; we solely rely on microcontrollers, making the optimization challenge even more formidable. These microcontrollers pose unique constraints, intensifying the need for advanced optimization techniques like distillation and quantization. Model compression, which aims to reduce the size of specific networks while preserving accuracy, has become increasingly vital in addressing the challenges posed

by resource-constrained environments.

Distillation approach involves training a compact model (the “student”) with minimal resource demands based on the probability distributions (logits) produced by a larger model (the “teacher”) for a specific task. These logits are often described as the "dark knowledge" possessed by the teacher model and serve as an extra source of information for the student’s training. Working with these softer targets is more manageable for the student model and has been demonstrated to enhance the ultimate accuracy of the student model.

This knowledge transfer is achieved through the use of a temperature parameter set to 5 and an alpha value of 1. This process enables the student model to emulate the performance of the teacher model while being more computationally efficient.

The use of dropout layers between each component of the architecture ensures regularization and helps prevent overfitting, contributing to the model’s robustness and its capacity to accurately classify various swimming types.

3.4 Network Input/Output

In order to feed our network, we gather data from real swimmers doing a set of different swims, where each swimmer wear a sensor on the head. We record the state of the sensor at a certain time-stamp. The data generated by the sensor contains 18 columns from which the features used as input are extracted (see Figures 5 and 6).

4 Experiments

4.1 Recognition of Swimming style

The recognition of swimming style is done by classifying data into five classes: butterfly, freestyle, breaststroke, backstroke swimming and out. Therefore, to recognize when the swimmer is out of water and when the sensor do not have signal, “ *Out* ” was selected as a separate class to recognize in the classification process (the sensor does not pick up a signal when the fix is false otherwise fix is true).

Features were extracted from each acceleration and gyroscope channel: standard norm, magnitude, fusion of acceleration, gyroscope and orientation data, and normalization of acceleration. A total number of 16 features were calculated for our model.

4.2 Evaluation metrics

In order to assess the effectiveness of a classifier, it’s essential to establish evaluation strategies. One prevalent approach is to determine the accuracy of a classifier, which measures the proportion of instances correctly assigned to their respective classes. A common method for this involves dividing the labeled section of the dataset into training and test data. The training data is employed for constructing the model, while the test data is utilized to gauge the model’s performance. The model’s predictions on the test data are subsequently compared to the correct labels, allowing for the calculation of the classifier’s accuracy.

In a multi-class problem, as in the stroke classification problem, this can be simplified

into a binary decision problem, as each instance can be classified as being in a given class or not. To evaluate the effectiveness of our model, we employ a confusion matrix, a tabular representation that succinctly outlines the model’s predictions in comparison to the actual class labels. This matrix offers a comprehensive breakdown, including true positives, true negatives, false positives, and false negatives.

Additionally, we utilize the recall metric, which gauges the model’s capability to identify all relevant instances by calculating the ratio of true positive predictions to the total actual positives.

Furthermore, the F1 score, serving as the harmonic mean of precision and recall, furnishes a well-balanced metric, capturing a nuanced assessment of the model’s performance in terms of both precision and recall.

4.3 Tests to improve accuracy

In order to improve accuracy, we develop four versions as follow : The first version contains rolling windows and quantization, the second one contains only quantization, the third version contains rolling windows, distillation without quantization and the last version contains rolling windows, distillation and quantization of the student model. All the versions are trained with the same model using a training set with Motion Windows using 16 features as input except the second version. We used “*adam*” as our optimizer, “*categorical cross entropy*” as our loss function for all versions. We initialized our neurons with zero values. Our model was trained using different epochs and batch size, we experimented increasing the number of epochs, but there was no difference in performance as it converged in the 34th epoch. The training took around more or less than 30 minutes to finish depending on the version used.

The first version use only quantization as compression model with rolling windows. The model was trained using 50 epochs and a batch of size 20. We use 3 LSTM layers with 50, 50 and 40 neurons respectively, and two dense layers with 40 and 5 neurons respectively. The first dense layer use “ReLU” function and the second one use “Softmax” as activations. Quantization of weights and activations is done on each layer with dropout for regularizations between each one.

In the second version, compression relies solely on quantization without using rolling windows. The model underwent training over 50 epochs with a batch size of 15. It is structured with three LSTM layers, each comprising 50, 50, and 40 neurons, and two dense layers with 40 and 5 neurons, respectively. The first dense layer employs the “ReLU” activation function, while the second layer uses “Softmax” as the version above. Quantization of weights and activations is performed at each layer, with the inclusion of dropout for regularization between them.

In the third version, we introduce distillation as the compression method, in addition to the use of rolling windows. Both the teacher and student models undergo training for 50 epochs, with a batch size of 32. The teacher model is designed with five LSTM layers, where the first layer consists of 50 neurons, and the subsequent layers each contain 40 neurons. Additionally, there are two dense layers, one with 40 neurons and the other with 5 neurons.

The student model, which is more simpler than the teachear model, is structured with

four LSTM layers, featuring 30, 20, 20, and 10 neurons, respectively, and two dense layers, one with 10 neurons and the other with 5 neurons.

In both models, the first dense layer applies the “ReLU” activation function, while the second layer utilizes “Softmax,” as described in the previous version. Notably, no quantization is introduced in this particular version. The knowledge transfer from the teacher model to the student one is achieved through the use of a temperature parameter set to 5 and an alpha value of 1.

The last version we use rolling windows and we introduce both distillation and quantization as the compression method. The quantization is only applied to the simpler model (student model). Both the teacher and student models undergo training for 50 epochs, with a batch size of 32. The teacher model is designed with four LSTM layers, where the first three layer consists of 50 neurons, and the last one of 40 neurons. Additionally, there are two dense layers, one with 40 neurons and the other with 5 neurons.

The student model, which is more simpler than the teacher model, is structured with four LSTM layers, the first one with 30 neurons, the next two with 20 neurons and the last one with 10, and two dense layers, one with 10 neurons and the other with 5 neurons.

In both models, the first dense layer applies the "ReLU" activation function, while the second layer utilizes "Softmax," as described in the previous version.

More precisely, the scenario is as follows: we have a state-of-the-art deep model, the “teacher,” trained to solve a specific task, and a compressed “student” model. The student model is compressed in two ways: first, it has a shallower architecture than the teacher, and second, it undergoes quantization, meaning its weights and activations are expressed with limited bit width. The strategy, similar to standard distillation methods, aims for the student to leverage the knowledge from the well-trained teacher model to achieve comparable accuracy. Notably, distillation has been previously employed to obtain compact high-accuracy representations of model ensembles [9], but we believe this is the first instance of its use for model compression through quantization.

4.4 Results

In order to assess the effectiveness of a classifier, it’s essential to establish evaluation strategies. One prevalent approach is to determine the accuracy of a classifier, which measures the proportion of instances correctly assigned to their respective classes. A widely adopted approach for this task entails partitioning the labeled segment of the dataset into training and test sets. The training data is utilized to build the model, while the test data is employed to assess the model’s performance. This process aids in evaluating the model’s ability to generalize to new, unseen examples. The model’s predictions on the test data are subsequently compared to the correct labels, allowing for the calculation of the classifier’s accuracy. Finally, the validation data is employed during the training phase to fine-tune the model’s parameters and prevent overfitting. It serves as an independent dataset that the model has not seen during training. In this work, we choose different CSV file for different swimmers to test performance.

Based on the results depicted in Figures 10 and 11, it is evident that versions 1,

3, and 4 yield the highest classification accuracy. In contrast, version 2 exhibits a poor performance in terms of loss (53.08%), despite achieving a reasonable accuracy (81.33%). Relying solely on quantization is not suitable for our work due to the utilization of data with time sequences. The characteristics of these sequences require the inclusion of rolling windows. Consequently, employing rolling windows in versions 1, 3, and 4 yields superior results compared to not using them as shown in Figures 10 and 11.

Model	Accuracy %	Loss
Version 1 : Rolling window with quantization	97.97	0.0863
Version 2 : Quantization	81.33	53.08
Version 3 : Rolling windows and distillation	98.13	0.0949
Version 4 : Rolling windows, distillation and quantization	98.56	0.1405

Figure 10: Comparative Results of our different versions in term of accuracy and loss.

Model	Val_accuracy %	Loss_accuracy
Version 1 : Rolling window with quantization	98.89	0.0521
Version 2 : Quantization	53.57	2.9425
Version 3 : Rolling windows and distillation	96.42	0.0827
Version 4 : Rolling windows, distillation and quantization	98.56	0.0694

Figure 11: Comparative Results of our different versions in term of val accuracy and val loss.

We observe that the accuracy and validation accuracy in version 2 is acceptable. However, upon examining the loss, a significant increase is noted, primarily attributable to the absence of rolling windows to handle time sequences. Analyzing the tables presented in Figure 10 and 11, it becomes evident that incorporating quantization is more effective in achieving favorable accuracy and loss results in the model.

We observe that our approach demonstrates effectiveness by comparing accuracy and validation accuracy. It is noteworthy that, in all versions except for version 2, the validation accuracy surpasses the accuracy. This observation is indicative of a positive sign, suggesting that our model is not over-fitting the data.

The accuracy of each version varies from one simulation to another due to the introduction of dropout between layers for regularization purposes. However, the differences in accuracy between simulations are relatively small, and the accuracy consistently remains above 95%.

Accuracy %	Version 3	Version 4
Teacher model	97.82	96.60
Student model	98.13	98.56

Figure 12: Comparative results between teacher and student model accuracy.

Val_accuracy	Version 3	Version 4
Teacher model	96.30	98.02
Student model	96.42	98.56

Figure 13: Comparative results between teacher and student model validation accuracy.

The strategy, as for standard distillation [1, 11] is for the student to leverage the converged teacher model to reach similar or higher accuracy.

The student in version 4 is compressed in the sense that it is shallower than the teacher; and it is quantized, in the sense that its weights are expressed at limited bit width. For the versions 3 and 4, as depicted in Figures 12 and 13, it is evident that the student model outperforms the teacher model. The student model, characterized by a simpler architecture compared to the teacher model with fewer layers and neurons, demonstrates highly satisfactory accuracy and loss. It is evident that incorporating quantization into the student model results in a higher accuracy compared to the model without quantization.

In version 3, the model incorporates rolling windows and distillation techniques, resulting in a commendable accuracy of 98.13% . However, in version 4, where both rolling windows, distillation, and additional quantization are employed, the accuracy is slightly higher at 98.56% . The introduction of quantization in version 4 specifically focuses on compressing the model’s parameters, aiming to reduce its overall size.

Quantization involves representing the model’s weights and biases with a lower number of bits, effectively compressing the information while maintaining a reasonable level of accuracy. This process is beneficial, especially in resource-constrained environments, as it leads to a more memory-efficient model that can be deployed on devices with limited capacity, such as microcontrollers.

The marginal increase in accuracy from version 3 to version 4 indicates that the quantization process has not significantly compromised the model’s ability to accurately classify swimming types. It showcases the balance achieved between model compression and accuracy improvement. This trade-off is crucial when considering real-world applications where computational resources are limited, and the need for efficient, high-performing models is paramount.

In summary, while version 3 demonstrates a strong accuracy, the slight improvement observed in version 4, achieved through the incorporation of quantization, underscores the effectiveness of combining various optimization techniques for enhancing both model efficiency and accuracy in the context of swimming type classification.

Despite the marginal differences in accuracy and validation accuracy between the two models, the gains in terms of running time and memory are noteworthy. This holds particular significance for us, considering that we intend to deploy this model in sensors where preserving minimal memory space is crucial.

Performance metrics play a crucial role in assessing and contrasting various classification models or machine learning techniques. Numerous metrics are indispensable for assessing the effectiveness of any multi-class classifier. These metrics serve two primary purposes: i) facilitating comparisons between the performances of distinct models, and ii) analyzing the behavior of a single model by adjusting different parameters. Many of these metrics draw upon the Confusion Matrix, as it encapsulates comprehensive information regarding the algorithm’s performance and classification rule. The confusion matrix serves as a tabular representation documenting the frequency of occurrences between two evaluators: the true/actual classification and the predicted classification. For uniformity across the paper, the columns signify the model’s predictions, while the rows indicate the actual classifications. The order of classes in both rows and columns remains consistent. Consequently, correctly classified instances align along the main diagonal, extending from the top left to the bottom right, representing the instances where the two evaluators are in agreement. To conduct a more in-depth analysis of these outcomes, we generated a Confusion Matrix. Upon examination, it becomes apparent that for versions 1, 3, and 4, all classes attained an accuracy rate of 98%.

Conversely, for version 2, the confusion matrix indicates poor performance as shown in Figure 14. The confusion matrix shows the highest values in the last column “Class *Out*”, the recall and F1 scores for the other classes are zero. It implies that the model is performing well in correctly identifying instances belonging to Class *Out*, but it is failing to recognize instances from the other classes.

In Figures 15 and 16, versions 3 and 4 show good performance for confusion matrix. The diagonal appears clearly in the confusion matrix showing the efficient of these two versions. The highest values in the confusion matrix are positioned along the diagonal, it signifies that the model has made accurate predictions across various classes (versions 1, 3 and 4). The diagonal elements represent true positives, indicating instances where the model correctly identified each class. The recall score for all classes is greater than 0.90 which implies that the model has effectively captured the

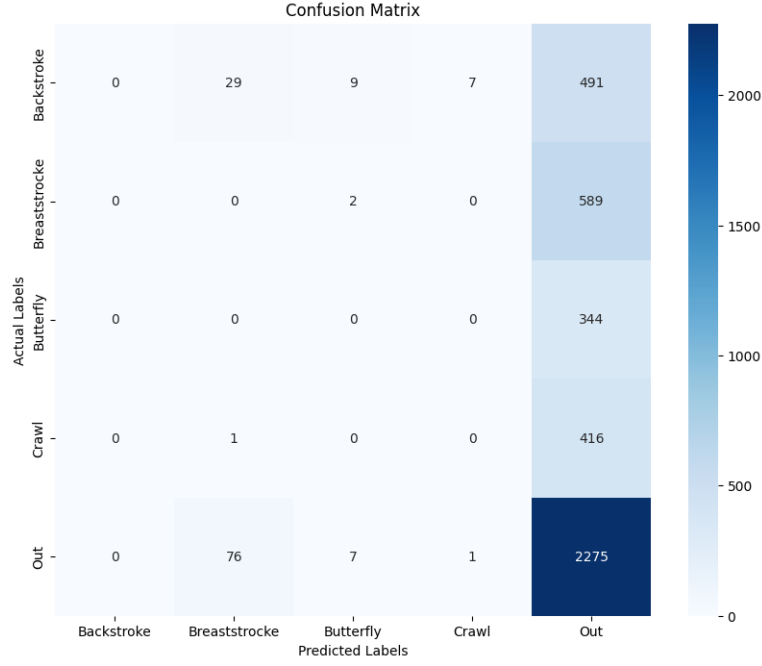


Figure 14: Confusion Matrix version 2

majority of positive instances within each class, showcasing its sensitivity to identifying relevant cases. Similarly, the F1 score exceeds 0.90 for all the classes, which balances precision and recall. This reflects a high level of overall performance in terms of both identifying positive instances and avoiding false positives. In essence, achieving high recall and F1 scores, coupled with a diagonal-heavy confusion matrix, suggests that the model is robust and proficient in correctly classifying instances across multiple classes, indicating a strong predictive capability with minimal errors.

Overall, the matrix provides a comprehensive overview of the model’s performance across different classes, enabling a nuanced assessment of its strengths and weaknesses in the multi-class classification task.

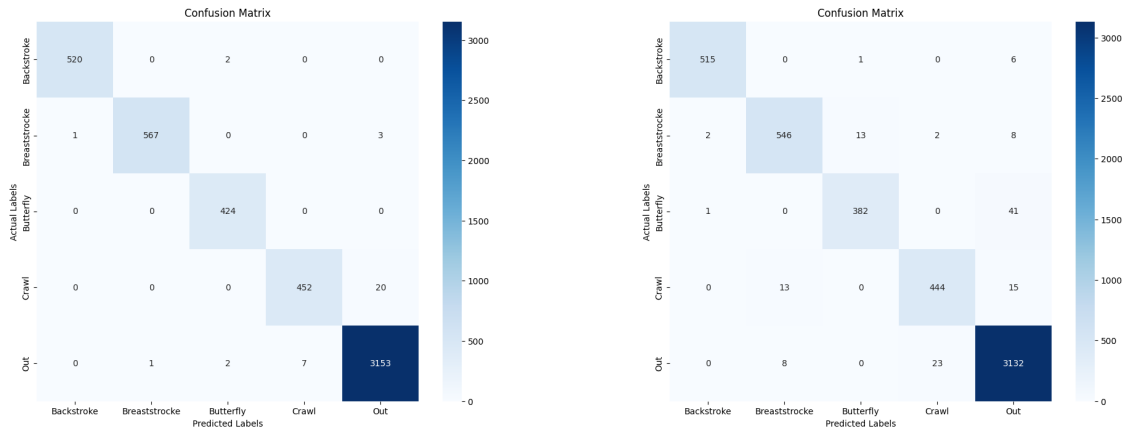


Figure 15: Confusion Matrix teacher vs student version 3

Finally, quantized distillation appears to be the method with best accuracy across the whole range of bit widths and architectures.

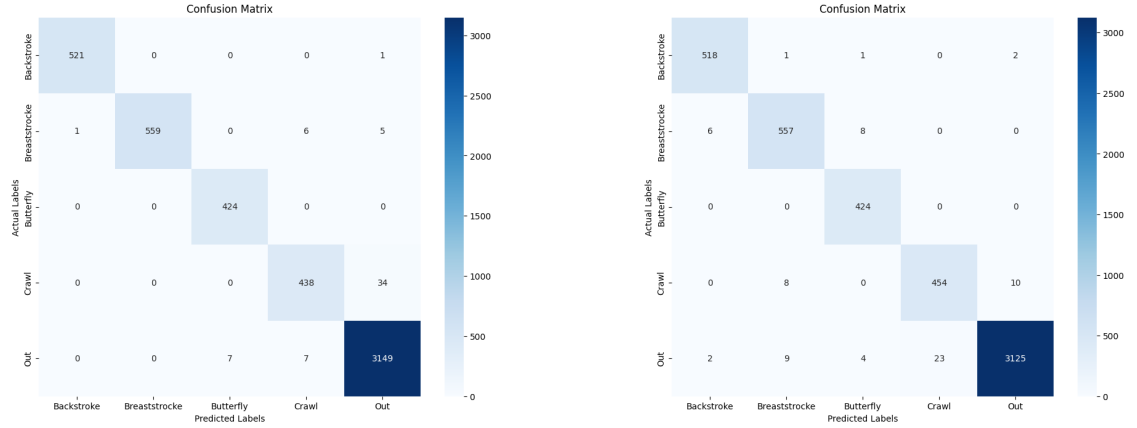


Figure 16: Confusion Matrix teacher vs student version 4

5 Conclusion

In the realm of multi-class classification for the recognition of five distinct swimming types, our comprehensive approach involving rolling windows, distillation, and quantization has yielded noteworthy results. The integration of these techniques has significantly enhanced the model’s ability to accurately discern intricate patterns associated with each swimming style.

The incorporation of rolling windows is important, particularly when dealing with time sequences inherent in swimming movements. The dynamic nature of swimming necessitates a meticulous analysis of temporal patterns, and the use of rolling windows ensures that the model captures these nuances effectively. This temporal sensitivity has proven instrumental in achieving a higher level of precision in the classification task.

Furthermore, the interaction between distillation and quantization has played a pivotal role in optimizing the model. Distillation, the process of transferring knowledge from a more complex model (teacher) to a simpler one (student), has enabled us to capture the essence of the complex swimming patterns in a more concise representation. This distilled knowledge is then coupled with quantization, which compresses the model’s parameters, resulting in a more memory-efficient framework.

The nuanced interplay between these techniques has not only facilitated model compression but has also contributed to the preservation of a commendable level of accuracy. This balance is crucial, especially in scenarios where computational resources are constrained, and the need for real-time processing is imperative.

In essence, our approach showcases the significance of addressing the temporal intricacies of swimming movements through rolling windows, while simultaneously optimizing model complexity through distillation and quantization. This multi-faceted strategy underscores the versatility of our model, making it well-suited for real-world applications where both accuracy and resource efficiency are paramount considerations. The successful classification of five swimming types stands as a testament to the efficacy of our approach and its potential applicability in diverse contexts within the field of activity recognition and beyond.

In addition to the intricate technicalities of our multi-class classification model,

a validation process has been integral in validating its performance and reliability. Employing a confusion matrix, recall, and F1 score has afforded us a granular understanding of the model’s predictive capabilities, enabling a comprehensive evaluation not just of overall accuracy but also of its proficiency in correctly classifying each specific swimming type. To fortify the robustness of our results, a diverse array of data from various swimmers has been carefully employed for testing. This multi-subject evaluation is essential in ensuring that our model generalizes well across different individuals, accounting for inherent variations in swimming styles among diverse participants. Subjecting our model to this comprehensive range of data sources instills confidence in its efficiency and applicability in real-world scenarios.

The incorporation of these performance evaluation metrics, coupled with exhaustive testing on varied datasets, contributes to the holistic validation of our multi-class classification model. This meticulous approach not only underscores the model’s accuracy and efficiency but also positions it as a reliable and adaptive solution for the nuanced task of recognizing distinct swimming types in a real-world context.

Moreover, the culmination of this work marks the inception of a broader objective - the exploration of miniaturized devices for real-time applications in sports analytics. Our future work aims to streamline and optimize the model for deployment on compact devices, ensuring portability and ease of integration into athletic training environments. The potential implications extend beyond swimming, offering opportunities for similar applications in various sports and physical activities.

In essence, this study not only contributes to the advancement of activity recognition technology but also sets the stage for practical implementations that can positively impact the sports industry. By transitioning towards miniaturized devices, we envision a future where athletes and coaches can access real-time insights, fostering continuous improvement and optimal performance in the dynamic world of sports.

Acknowledgements

The authors extend their thanks to the athlete Alizée Pelletier for her contributions in integrating theoretical concepts with practical swimming techniques. Utilizing her Junior skills in mathematics and engineering, she has effectively conducted measurements and analyses within the swimming team at Florida Southern University (USA) and the Viking Team at Rouen (France). Furthermore, the authors would like to acknowledge Nicolas Leday for his indispensable role in adapting the Lara device, a crucial tool for data collection in this study. Additionally, we value his insightful discussions on utilizing the device and exporting the results, which significantly enhanced the methodology and outcomes of this research.

References

- [1] BA, J., AND CARUANA, R. Do deep nets really need to be deep? *Advances in neural information processing systems* 27 (2014).
- [2] CHEN, W., WILSON, J., TYREE, S., WEINBERGER, K., AND CHEN, Y. Compressing neural networks with the hashing trick. In *International conference on machine learning* (2015), PMLR, pp. 2285–2294.

- [3] COSTA, J., SILVA, C., SANTOS, M., FERNANDES, T., AND FARIA, S. Framework for intelligent swimming analytics with wearable sensors for stroke classification. *Sensors* 21, 15 (2021), 5162.
- [4] DATAR, M., AND MOTWANI, R. The sliding-window computation model and results. In *Data Stream Management: Processing High-Speed Data Streams*. Springer, 2016, pp. 149–165.
- [5] DITZLER, G., ROVERI, M., ALIPPI, C., AND POLIKAR, R. Learning in nonstationary environments: A survey. *IEEE Computational Intelligence Magazine* 10, 4 (2015), 12–25.
- [6] GONG, Y., LIU, L., YANG, M., AND BOURDEV, L. Compressing deep convolutional networks using vector quantization. *arXiv preprint arXiv:1412.6115* (2014).
- [7] GOODFELLOW, I., BENGIO, Y., AND COURVILLE, A. *Deep learning*. MIT press, 2016.
- [8] GRAVES, A., AND GRAVES, A. Long short-term memory. *Supervised sequence labelling with recurrent neural networks* (2012), 37–45.
- [9] HINTON, G., VINYALS, O., AND DEAN, J. Distilling the knowledge in a neural network. arxiv e-prints, page. *arXiv preprint arXiv:1503.02531* 3 (2015).
- [10] LECOUTERE, J., AND PUERS, R. Wireless communication with miniaturized sensor devices in swimming. *Procedia Engineering* 72 (2014), 398–403.
- [11] LECUN, Y., BENGIO, Y., AND HINTON, G. Deep learning. *nature* 521, 7553 (2015), 436–444.
- [12] MICHAELS, S., TAUNTON, D. J., FORRESTER, A. I., HUDSON, D. A., PHILLIPS, C. W., HOLLISS, B. A., AND TURNOCK, S. R. The use of a cap-mounted tri-axial accelerometer for measurement of distance, lap times and stroke rates in swim training. *Procedia engineering* 147 (2016), 649–654.
- [13] WOLLMER, M., EYBEN, F., KESHET, J., GRAVES, A., SCHULLER, B., AND RIGOLL, G. Robust discriminative keyword spotting for emotionally colored spontaneous speech using bidirectional lstm networks. In *2009 IEEE International Conference on Acoustics, Speech and Signal Processing* (2009), IEEE, pp. 3949–3952.
- [14] ZHOU, A., YAO, A., GUO, Y., XU, L., AND CHEN, Y. Incremental network quantization: Towards lossless cnns with low-precision weights. *arXiv preprint arXiv:1702.03044* (2017).